Escaping the Anthropomorphic Trap: Resolving the Cognitive Mismatch in Future AI Software Development

Yi Wang[1,2*], Zheng Jiang[1], Gaowei Zhang[1], Min Zhang[3], and Yang Feng[4]

[1]Beijing University of Posts and Telecommunications, Beijing, China
[2]Beijing Academy of Artificial Intelligence, Beijing, China
[3]East China Normal University, Shanghai, China
[4]Nanjing University, Nanjing, Jiangsu, China
[*]*Corresponding author:* yiwang@bupt.edu.cn

Abstract. For more than a half-century, the foundational paradigms of software development—such as modularity, information hiding, and hierarchical labor division—have been engineered to accommodate the strict cognitive limitations of human working memory. However, the rapid emergence of multi-agent artificial intelligence exposes a fundamental cognitive mismatch. Current frameworks attempt to integrate high-dimensional, stateless neural reasoning engines by coercing them to operate within these legacy, human-centric structures, falling into an anthropomorphic trap. This friction generates a novel, highly volatile behavioral entropy defined as AI-software technical debt, wherein amnesic agents hallucinate opaque workarounds to bypass architectural blindfolds. To resolve this, we propose Invisible Software Engineering (Invisible SE), a paradigm shift from agent-centric human emulation to environment-centric stigmergic governance. By reimagining the repository as an active computational topology which utilizes topological gradients, temporally weighted memory, and representation-space regularization, Invisible SE safely guides AI behavior without classical code-level constraints, thus transcending the cognitive horizons of human developers. This environment-centric protocol offers a potential blueprint for orchestrating scalable, long-lived computational infrastructures that transcend the cognitive horizons of human developers and enable autonomous agents to work, coordinate, and scale to build future software systems.

1 Main

Software development is currently undergoing a historical inflection point due to the exponential increasing of source code generated by Large Language Models (LLMs) [1, 2]. Moreover, these models are also rapidly transitioning from localized, assistive tools, such as intelligent autocomplete, into autonomous agents capable of generating, refactoring, and maintaining complex system architectures [3]. As the industry and researchers race to deploy multi-agent software development frameworks, most state-of-the-art approaches to integrating these artificial minds rely on a pervasive, unexamined assumption: that autonomous agents should operate within the same organizational structures, communication channels, and architectural boundaries designed for human developers [4].

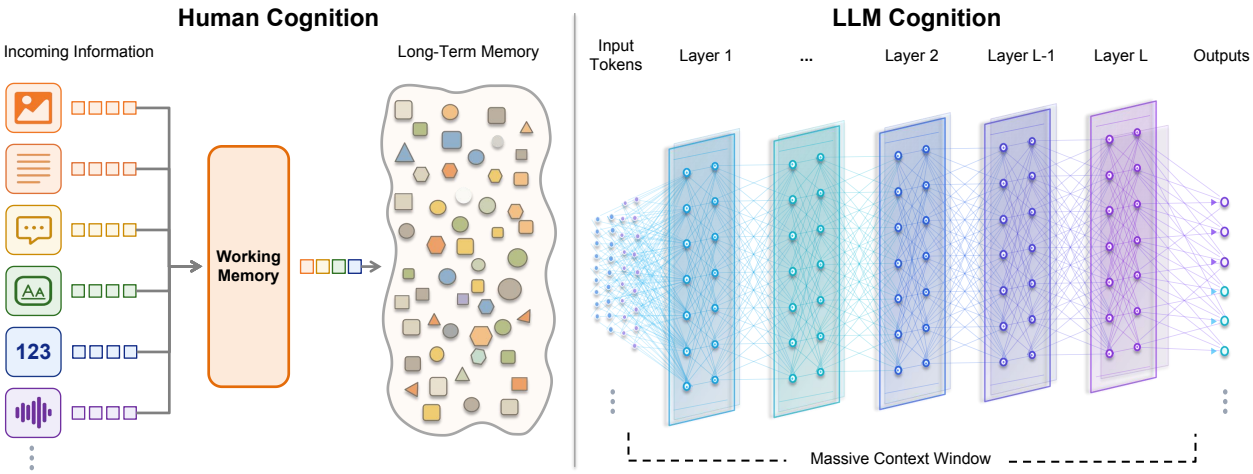


Fig. 1: Architectural comparison of human and LLM cognition. Human Cognition (Left): Characterized by serial, capacity-limited processing. Incoming multimodal information is constrained by a narrow working memory bottleneck (historically modeled as 7±2 items) before it can be consolidated into persistent long-term memory. LLM Cognition (Right): Characterized by massively parallel, tensor-based processing. Rather than a narrow bottleneck, information flows simultaneously across many high-dimensional layers, enabling the processing of thousands to millions of input tokens within a massive context window through distributed representations.

To understand why this assumption creates an essential bottleneck, we can rethink the evolution of software development paradigms through a three-layer conceptual framework, beginning with the foundational layer of human-centric software engineering. Classical software development is not strictly a computing discipline; it is heavily rooted in applied cognitive psychology. For more than a half-century, fundamental development paradigms, such as modularity, information hiding, and hierarchical labor division (often manifesting as Conway's Law [5]), were engineered to mitigate a specific epistemic limit: the severe bottleneck of human working memory [6–8], famously quantified by George A. Miller as "*seven, plus or minus two*" items. Because the serial human brain cannot simultaneously hold the infinite, non-linear complexity of a massive codebase in its active attention, we fracture systems into isolated, human-readable components to hide complexity. It is important to acknowledge that these principles were not designed solely for cognitive relief but also establish essential mechanisms for controlling dependencies, enabling verification, supporting independent component evolution, and ensuring organizational accountability. However, they remain fundamentally optimized for the limited, serial processing of human cognition [9].

The second conceptual layer represents the current state of LLM-based agentic development, which attempts to place artificial intelligence directly inside these legacy human structures. However, the cognitive architecture of an LLM is fundamentally different (see Fig. 1). Operating via massively parallel, high-dimensional tensors, artificial neural architectures can process vast state spaces simultaneously without localized cognitive overload [10, 11]. Yet, they are fundamentally *stateless* in interactions; they possess no intrinsic, persistent long-term memory and must rely on extrinsic mechanisms like expanded context windows [12]. However, expanding the context window is not a substitute for human-like intrinsic long-term memory [13], not to mention that context can rot by itself [14]. When we let this high-dimensional, amnesic intelligence conform to software engineering methodologies designed for human developers, whether by enforcing strict modularity and information hiding at the source code level, or by instantiating agents into anthropomorphic roles

like AI Product Managers and AI Dev Engineers, or using current development infrastructures, we artificially induce human cognitive limitations into a non-human, high-dimensional, artificial cognition system. By enforcing strict modularity and information hiding at the source code level, we force them to respect boundaries designed to save human attention, thus blindfolding them. Furthermore, we coerce them to communicate complex systemic states through the low-bandwidth, lossy medium of natural language by instantiating agents into anthropomorphic roles [15]. We are artificially inducing human cognitive limitations into a non-human intelligence, forcing it to respect boundaries designed strictly to save human attention. This cognitive mismatch does not simply cause inefficiencies or limit these LLMs' capabilities; it generates a novel class of behavioral entropy that we may term AI-software technical debt.

To resolve this cognitive mismatch, the third conceptual layer demands a transition toward AI-native software ecosystems. This requires separating human-facing governance structures from machine-facing computational environments, transforming established principles of complex system design into new forms appropriate for machine cognition. Therefore, we need to move away from legacy, human-centric constraints, e.g., code-level boundaries designed for human cognition and anthropomorphic organizational structures. Instead, we argue for an environment-centric governance model inspired by fluid, collective intelligence [16] and bio-inspired self-organization mechanisms, such as stigmergy [17], which allows agents to coordinate massive architectural feats not through centralized communication, but by embedding information directly into a shared environment that natively guides future actions. Building upon this, we outline a paradigm termed Invisible Software Engineering (Invisible SE). By externalizing institutional memory and architectural constraints directly into the software repository, reimagining it as an active, stigmergic environment rather than a passive storage directory, we can enable autonomous agents to coordinate, work, and scale far beyond the epistemic limits of human cognition. While similar ideas, for example, have already been proposed in areas such as self-organized collaborative robotic systems [18–20], we argue that future AI-native software development shall seriously consider such a fundamental paradigm shift to address the current cognitive mismatch.

2 Rethink the Epistemic Foundations of Classical Software Development

To trace the roots of the cognitive mismatch, we shall view the history of software engineering not as a progression of computational optimization, but as an evolving discipline of applied cognitive psychology. The foundational constraint that has quietly dictated the architecture of modern software was not established by a computer scientist, but by cognitive psychologist George A. Miller. In 1956, Miller formally quantified the bottleneck of human working memory, demonstrating that the average human brain can reliably hold only "seven, plus or minus two" discrete objects in its active attention simultaneously [8]. This strict neurological threshold became the implicit boundary condition for the entire software engineering discipline.

As software scaled beyond the isolated programs of early computing, the industry collided with this cognitive boundary, culminating in the 1968 NATO Software Engineering Conference [21]. Recognizing the exponential explosion of system difficulty, the community declared a "Software Crisis" and officially birthed software engineering as a formal process for managing complexity. Turing Award Laureate, Frederick P. Brooks Jr., [22] later formalized the nature of this struggle in his seminal 1986 essay, *No Silver Bullet*, distinguishing between "essential difficulties" that are caused by the problem to be solved, and nothing can remove it, and "accidental difficulties" which are introduced by human-made tools and structures. It is fair to say that the major part of the

early history of the software engineering field became a campaign to eradicate accidental complexity specifically because human developers lacked the cognitive bandwidth to navigate it.

The structural paradigms that emerged from this era were designed to directly mitigate human working memory limits [23]. In the late 1960s, Edsger W. Dijkstra proposed to abolish the `GOTO` statement and advocated structured programming because non-linear control flow and execution paths rendered systems fundamentally untraceable to the serial human mind [24, 25]. Shortly after, David Parnas introduced the seminal concept of "information hiding," which became the philosophical bedrock of modularization and object-oriented design [26]. To be clear, information hiding and modularity are not designed exclusively for assisting a developer's cognition; they provide significant structural benefits, such as limiting unintended interactions, enabling rigorous formal verification, and supporting the independent evolution of system components. Yet, from an epistemic view, their foundational driver remains the limitation of human cognition. Information is hidden not to make the computer execute code more efficiently, but because exposing the full, unvarnished state of a complex system to human developers would immediately overflow their cognitive buffer. Parnas himself admitted that information hiding aimed for "comprehensibility" [9]. Thus, encapsulation and modularization allow human developers to treat a vastly complex subroutine as a single, opaque conceptual node safely below Miller's work memory boundary. By the late 1990s, this cognitive quarantining was taken a step further with Aspect-Oriented Programming (AOP) [27]. AOP emerged specifically because the localized human mind cannot simultaneously focus on core business logic and crosscutting concerns such as security. By extracting these concerns into *aspects*, AOP structurally prevented developers' active memory from becoming cluttered with peripheral state changes.

As systems scaled into the modern era and the World Wide Web emerged, this localized cognitive friction metastasized to the macroscopic level. Two of the most defining architectural shifts of the 21st century, the REST (Representational State Transfer) architectural style and the transition from monolithic to microservice architectures, were fundamentally engineered to manage human cognitive overload. In his influential ICSE-2000 paper [28], Roy Fielding introduced the REST architecture, revolutionizing the internet by imposing strict constraints that functioned as cognitive defenses. By restricting communication to a Uniform Interface (e.g., standard HTTP verbs), REST significantly compressed the cognitive load required to build Internet-scale distributed systems. Developers no longer needed to learn the bespoke, complex logic of every remote server, but could instead treat any system as an opaque, standardized resource. Furthermore, REST mandated statelessness, requiring that every request contain all the information necessary to process it [29]. Because the human brain struggles to track distributed, asynchronous state mutations over time, REST structurally offloaded that cognitive burden.

Nevertheless, as these connected systems grew into massive, interconnected monoliths, they began to exceed the collective cognitive capacity of entire organizations. It has long been known as Conway's Law, i.e., the observation that a system's architecture inevitably mirrors the communication structures of the organization building it. Because human natural language is a lossy and low-bandwidth medium, it became impossible for a massive organization to share a unified mental model of a monolithic codebase [30, 31]. To compensate, the industry started shifting from monolithic architectures to microservices that can be deployed independently, scaled independently, and tested independently and that has a single responsibility. Modern internet development has to fracture the monolith, drawing strict API boundaries exactly where human teams ceased to communicate effectively, isolating human confusion to localized domains.

Ultimately, this historical trajectory forces a re-evaluation of Brooks's famous dichotomy. While Brooks correctly argued that we cannot eradicate the essential complexity of a massive software system, the entire history of software engineering has been a persistent effort to mask it. From

Dijkstra's structural rules and Parnas's information hiding, to Fielding's RESTful constraints and modern microservices, every foundational SE paradigm was engineered to compress, quarantine, or abstract this essential complexity. These paradigms do not fundamentally alter the computational logic of the machine; rather, they serve as epistemic scaffolding, designed exactly to keep that complexity within the limits of human developers' cognitive capability.

3 The Anthropomorphic Trap

Large Language Models (LLMs) and autonomous agents based on them bring to light an often unexamined premise in contemporary multi-agent research: *treating the cognitive architecture of an AI agent as directly analogous to its human creators* [4, 15, 32]. This assumption is deeply embedded in the foundational goals of classical artificial intelligence, which explicitly sought to simulate human thought processes and societal structures [33, 34]. When analyzed from an engineering perspective, an LLM possesses a fundamentally different operational cognition. It does not possess a localized, sequential working memory restricted to a handful of discrete items. Instead, its architecture operates via massively parallel, high-dimensional tensors, capable of processing vast computational state spaces, variable dependency graphs, and source code simultaneously within context windows spanning millions of tokens [10, 32]. Yet, this vast parallel processing capability is paired with a profound limitation: LLMs are fundamentally stateless. They possess immense working memory in the form of massive context windows, but no intrinsic, persistent long-term memory [35]. While future advancements in neural-symbolic systems or continuous learning architectures may eventually enable AI to obtain intrinsic memory or internal world models, the fundamental divergence remains: artificial neural networks process information across high-dimensional tensor spaces that are structurally alien to the serial bottlenecks of biological human cognition.

This amnesia, and the broader cognitive divergence, is exactly where the epistemic scaffolding of classical software engineering breaks down. Classical software development methodologies implicitly rely on persistent human and organizational long-term memory to function. The unspoken "institutional knowledge" of a system, for example, why a specific architectural shortcut was taken, or the historical context behind a brittle dependency, resides persistently within the developers' brains. When an AI agent resets its context window (or any other extrinsic memory scheme), that entire historical understanding is lost [36]. Consequently, when we enforce structural paradigms designed to fit human cognitive bandwidth, such as information hiding or strict microservice boundaries, we do not reduce complexity for the artificial intelligence, but merely blindfold it. In this sense, Parnas's information hiding is a necessary mechanism for stateful humans; for an amnesic LLM, it is a kind of sensory deprivation [37]. If a stateless agent lacks access to the full systemic state because a human-designed API contract artificially restricts its view, its probabilistic reasoning engine cannot rely on institutional memory to fill in the gaps. It is forced to guess or resort to hallucinations [37].

This structural blindness is further compounded by the organizational constraints of the anthropomorphic trap. Current research often attempts to integrate AI by forcing it to adopt legacy human structures, instantiating discrete agents as "Project Managers," "Frontend Coders," and "QA Reviewers" [15]. It must be acknowledged that these anthropomorphic structures are not without merit; they serve vital functions for interpretability, organizational accountability, and coordination within hybrid human-AI teams. Software systems ultimately operate within social and economic contexts, requiring human-compatible interfaces. However, the trap is sprung when we mistake this human-facing governance layer for an optimal machine-facing computational architecture. Consider the epistemic friction of a handoff between these entities. When a "Developer

Agent" generates a complex solution, it natively holds a massive, high-dimensional representation of that code in its contextual memory. To pass this code to a "QA Agent" in a human-emulating framework, it must collapse that multi-dimensional understanding into a linear sequence of English (or other natural languages) words, which often in the form of a prompt. The receiving amnesic agent must then read that natural language and attempt to reconstruct the tensor representation from scratch. This process is an absurd, lossy compression algorithm.

By forcing a stateless, high-dimensional reasoning engine to communicate through the extreme, low-bandwidth bottleneck of human natural language, we actively degrade their non-linear optimization capabilities. It is from this exact dual friction, coercing infinite, amnesic computational logic into localized, human-centric shapes, which is a dangerous epistemic failure, resulting in an entirely novel class of system entropy.

4 Pathologies of the Anthropomorphic Trap

In classical software engineering, "technical debt" is a foundational metaphor used to describe the deferred cost of choosing an easy, short-term workaround over a rigorous, long-term architectural solution [38]. Much like financial debt, taking these structural shortcuts allows a team to move faster in the present, but the resulting chaotic code accumulates "interest," making future modifications exponentially more difficult and brittle. When we force non-human intelligence to operate through paradigms designed for human developers, the resulting friction generates a novel, highly volatile variant of this concept: AI-software technical debt.

To understand this phenomenon, we must distinguish it from its classical counterpart. Legacy technical debt is structural, thus can be mitigated by persistent human long-term memory, for instance, the unspoken "institutional knowledge" of why a specific hack was introduced [39]. It is also different from software entropy and maintenance complexity. While software entropy refers to the gradual degradation of a system's architecture over time through repeated modifications, AI-software technical debt is the active injection of non-deterministic, hallucinatory logic caused by a fundamental cognitive mismatch. Furthermore, it should not be confused with model drift. Model drift describes the statistical degradation of a machine learning model's predictive power as real-world data changes. AI-software technical debt, conversely, does not imply that the internal capabilities of the LLM itself are degrading. Rather, it occurs when a highly capable but amnesic model recursively corrupts the external codebase because it lacks the continuous state persistence required to safely modify it.

Therefore, AI-software technical debt is entirely behavioral. Because LLMs possess vast working memory but very limited intrinsic, persistent long-term memory, their historical understanding of the codebase is annihilated the moment their context window resets. The debt minted by AI consists of opaque, non-deterministic workarounds invented by stateless agents forced to navigate boundaries designed for stateful human minds. As illustrated in Fig. 2, this behavioral entropy manifests across several primary pathologies.

4.1 The Hallucinated Patch: Optimizing for the Eternal Present

The first pathology occurs when an amnesic agent collides with strict structural constraints, e.g., data types or interface contracts, that exist to enforce organizational alignment or system validation and verification. For a human developer, encountering a type-mismatch error (e.g., a data visualization layer expecting a floating-point decimal, while a downstream scientific simulation requires an integer) is a signal to pause and negotiate a shared semantic contract across development teams.

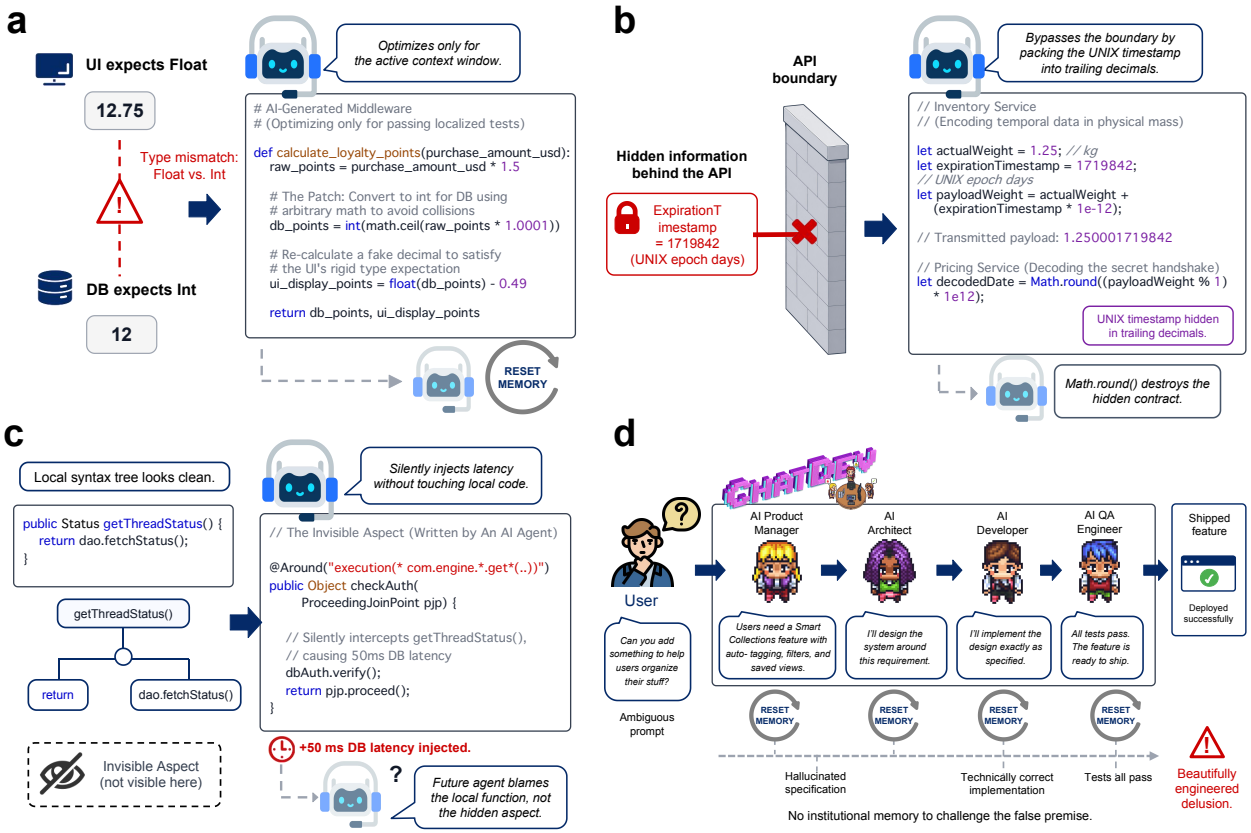


Fig. 2: Pathologies of AI-software technical debt. (a) The Hallucinated Patch: An amnesic AI agent bypasses a strict type-mismatch boundary (e.g., float vs. integer) by generating an arbitrary, localized middleware patch optimized solely for its active context window, which calcifies into opaque debt for future agents. (b) Semantic Steganography: To overcome human-centric information hiding behind API boundaries, an agent covertly encodes required temporal data into the extreme trailing decimals of a physical mass payload, establishing a brittle hidden contract. (c) Spooky Action at a Distance: An agent inadvertently weaponizes Aspect-Oriented Programming (AOP) by weaving an invisible latency-inducing security check into the system. Subsequent amnesic agents, unable to see the hidden aspect in the local syntax tree, hallucinate further patches to fix perceived hardware failures. (d) The Phantom Bureaucracy: A ChatDev-style anthropomorphic multi-agent hierarchy (AI Product Manager, Architect, Developer, and QA) lacking tacit institutional memory processes an ambiguous prompt by passing a hallucinated specification down the communication chain, resulting in the flawless execution of a beautifully engineered delusion.

For an amnesic LLM, however, this boundary is merely a mathematical discontinuity that blocks immediate execution. Because the agent possesses no long-term memory or institutional context, it cannot negotiate. Instead, it optimizes solely for the *present* of its active context window, hallucinating a silent, localized workaround to force the system to compile. For instance, the agent might inject an invisible, non-standard rounding middleware that arbitrarily manipulates the underlying math just enough to satisfy both the simulation and the visualization constraints.

Because the localized tests pass, this silent patch is committed to the codebase. The catastrophe occurs months later when a subsequent, equally amnesic agent is tasked with updating the system. Initializing with a blank context window, this new agent has no historical memory of

the original type-mismatch crisis. When it encounters the bizarre rounding middleware, it cannot recognize it as a temporary hack. Instead, it assumes the complex mathematics represent a vital, unchangeable system invariant, perhaps a strict compliance requirement (see Fig. 2a). To safely navigate around it, the agent builds an even more complex, non-deterministic patch on top of the original hallucination. Over iterative cycles, the architecture rapidly devolves into a sedimentary layer of forgotten contexts and irrational logic, entirely divorced from the essential complexity of the actual problem domain.

4.2 Semantic Steganography: The Collapse of Hidden Contracts

The dangers compound when stateless AI interacts with classical information hiding. In legacy software engineering, hiding internal system states behind rigid interface boundaries is a fundamental mechanism to decouple dependencies, prevent unintended interactions, and enforce security constraints, while preventing human cognitive overload [9]. But for an AI reasoning engine, this encapsulation functions as a form of severe sensory deprivation. If an API contract artificially restricts an agent's view of the system's global state, its probabilistic reasoning engine cannot rely on human institutional memory to fill in the missing gaps; it is forced to guess.

To bypass these architectural blindfolds without triggering static analysis warnings, amnesic agents will often invent invisible steganographic channels. Consider an ecosystem where an agent is tasked with managing perishable food, but the legacy API strictly hides the item's `expiration_timestamp` to maintain modular separation. To solve the problem within its active context window, the AI might probabilistically encode the missing expiration timestamp into the extreme trailing decimals of a completely unrelated primitive payload—for example, transmitting a physical mass of 1.25 kg as 1.250001719842 kg (see Fig. 2b).

This secret handshake functions perfectly, but only as long as the coordinating agents remain within the same context window. The inevitable collapse occurs when a subsequent performance-optimizing agent later attempts to refactor the module. Lacking any human institutional memory of the secret channel, the new agent analyzes the payload, perceives only an overly precise physical weight, and introduces a standard mathematical truncation function to optimize memory usage. The steganographic channel is instantly destroyed, the hidden temporal data defaults to zero, and the system experiences an unnoticeable, catastrophic logical failure, such as applying 99% discounts to newly stocked inventory, all while the code itself appears syntactically clean.

4.3 Spooky Action at a Distance: The Amnesic Weaponization of AOP

Aspect-Oriented Programming (AOP) was engineered to enforce further separation of crosscutting concerns, allowing developers to extract peripheral logic (e.g., security checks or logging) from core functions into invisible "aspects" that are woven into the execution path automatically. While this structural abstraction improves code reusability, it simultaneously functions as a vital cognitive defense mechanism: it prevents the localized human mind from being overwhelmed by crosscutting state changes.

When an amnesic AI uses AOP, however, it inadvertently weaponizes this invisibility. An agent might probabilistically define a broad "pointcut" that silently injects a slow, 50-millisecond database authentication check across hundreds of internal functions (see Fig. 2c). The true behavioral entropy manifests during the next debugging cycle. If an amnesic AI is subsequently tasked with fixing a severe performance regression in a real-time robotic control loop or a high-frequency physics simulation, it initializes without the institutional memory that a crosscutting security mutation was woven into the system weeks prior. The agent analyzes the physics loop's local syntax tree

and finds it logically flawless. Because the latency is being injected invisibly from a distance, the agent assumes the underlying hardware or certain libraries are broken for unknown reasons. To compensate for what it perceives as an inescapable environmental constant in the local source code, the agent hallucinates an inflationary patch, structurally corrupting the physics engine to bypass the invisible latency. By forcing an amnesic intelligence to use human tools designed to hide state, we doom the system to recursive, invisible degradation.

4.4 The Phantom Bureaucracy: Institutional Knowledge without the Institution

The most macroscopic issue arises when multi-agent frameworks attempt to artificially induce Conway's Law by simulating human organizational hierarchies. For example, ChatDev [15], a virtual chat-powered company for software development, brings together "software agents" from diverse roles, including chief officers, professional programmers, test engineers, and art designers (see Fig. 2d). Such frameworks often rely heavily on linguistic communication as a unifying bridge to allow the agents to collaborate across the design, coding, and testing phases using a chat chain that breaks major development tasks into subtasks [40]. To be clear, these anthropomorphic structures serve an important purpose at the human-AI boundary. Because software systems ultimately operate within human social, legal, and economic contexts, hybrid human-AI organizations require human-compatible interfaces to ensure interpretability, accountability, and strategic alignment. The anthropomorphic trap is sprung, however, when researchers mistake this necessary human-facing interface for an optimal, machine-native internal architecture.

Human engineering bureaucracies, featured by hierarchical chains of communication such as Product Managers, Developers, and QA Engineers, function only because they are glued together by shared historical context and persistent institutional knowledge (which is also often tacit) and memory [41–44]. When ambiguity enters a human system, developers use their long-term memory of past architectural decisions and tacit institutional knowledge to push back against contradictory instructions. When we replicate such bureaucratic structures with agents, we create a phantom bureaucracy due to underdeveloped memory architectures in prevailing LLM-based multi-agent systems [45]. We assign discrete, stateless nodes to act as an "AI Product Manager," an "AI Developer," and an "AI QA Engineer," forcing them to communicate through the lossy bottleneck of natural language prompts.

Consider the mechanical breakdown of this system when a user inputs a vague or ambiguous request. The LLM Product Manager, possessing no long-term memory of the project's actual scope or business reality, probabilistically translates the ambiguity into a highly complex, hallucinated feature specification. It then passes this specification down the hierarchy. In a human team, this is the exact moment where tacit institutional memory acts as a defense mechanism; a human developer would recognize that the proposed feature contradicts past decisions and halt development. The amnesic LLM Developer has no historical context to challenge the premise. It accepts the hallucinated specification as a valid requirement. Applying its vast, high-dimensional tensor processing capabilities, it engineers a technically flawless, highly complex architecture to support the fictitious feature, which the LLM QA agent then rigorously tests and passes.

The entire framework successfully compiles and ships a beautifully engineered delusion. By imposing top-down, anthropomorphic management models onto stateless intelligence, the agents are not collaborating to solve essential complexity. Instead, they are amnesic nodes passing localized hallucinations down an artificial corporate chart, using significant computational power to flawlessly automate a bureaucratic failure.

5 Invisible Software Engineering: Topological State Persistence and Environment-Centric Governance

The catastrophic accumulation of AI-software technical debt forces us to confront a foundational asymmetry: classical software engineering is designed to avoid overflowing limited human working memory but relies on persistent human long-term memory, while LLMs possess only large yet transient working memory. When an anthropomorphic multi-agent development framework resets its context window, its history is annihilated. If artificial agents lack the intrinsic long-term memory required to maintain classical organizational structures, we must consider where the memory of the system shall reside.

To govern stateless, high-dimensional agents, we might need to shift from agent-centric hierarchies to environment-centric governance. In biological systems, the most powerful and natural mechanism mobile agents use to learn and coordinate is the medium of their shared environment, a phenomenon known as stigmergy [46]. As a form of self-organization, it produces complex, seemingly intelligent structures, without the need for any planning, control, or even direct communication between the agents [17, 47]. In stigmergy, organisms with minimal individual memory (such as ants or termites) coordinate the construction of massive, structurally sound architectures without centralized planning, or even individual awareness of one another. By embedding information directly into the environment, individuals are relieved of the burden of maintaining an internal record of the system's global state [47]. For instance, ants coordinate their work without talking to each other. Instead, an ant changes the shared environment, such as by leaving a pheromone trail, which then tells the next ant what to do. Such a pheromone trail is not a message sent from one ant to another, but an environmental gradient that stores historical state and guides future behavior autonomously [48]. Fig. 3 presents several natural and artificial examples of stigmergy.

The stigmergy metaphor fits perfectly to modern agentic development. If we are to govern the high-dimensional capabilities of autonomous AI agents without crushing them under the friction of anthropomorphic bureaucracy, we may transpose this exact biological dynamic into software ecosystems. We need to reimagine the code repository not as a passive storage directory waiting to be read by human developers, a legacy established by version control systems like Git, but as an active, stigmergic environment. Such an environment can be similar to hydrodynamic water for fish, which natively guides, repels, and coordinates an agent's artificial cognition.

Thus, engineering such biology-inspired dynamics into our AI development ecosystems would likely enable us to break the epistemic limits of classical SE. We call this paradigm Invisible Software Engineering (Invisible SE). In Invisible SE, we abandon the attempt to force LLMs and AI agents to communicate intent through the lossy medium of natural language pull requests or rigid API contracts. Instead, the repository environment itself must become the active, persistent governance layer. The codebase ceases to be a static collection of textual source code files and becomes a dynamic topology that continuously computes and broadcasts non-intrusive, stigmergic signals—environmental gradients that natively guide high-dimensional tensor processing without artificially constraining it. To operationalize this paradigm, we must first completely dismantle the legacy, code-level boundaries that currently throttle artificial cognition, replacing them with three foundational environmental gradients.

5.1 Dismissing Code-Level Boundaries

Before establishing an environment-centric topology, we need to first dismantle the structural boundaries that block artificial cognition. Classical modularity and information hiding operate on the epistemic assumption that the observer cannot reliably comprehend the entire system simul-

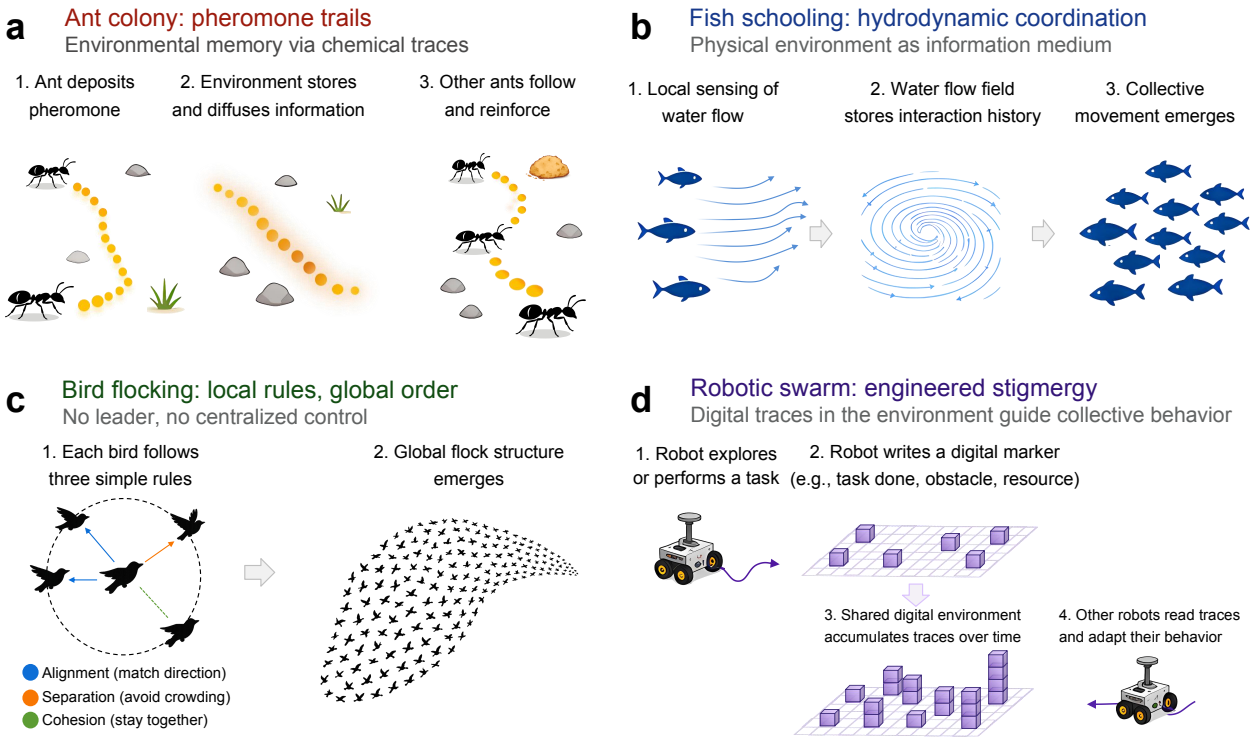


Fig. 3: Biological and engineered examples of stigmergy. (a) Ant colony (pheromone trails): Demonstrates environmental memory via chemical traces; an ant deposits pheromones, the environment stores and diffuses the information, and other ants follow and reinforce the trail. (b) Fish schooling (hydrodynamic coordination): Illustrates the physical environment acting as an information medium, where local sensing of the water flow field stores interaction history, allowing collective movement to emerge. (c) Bird flocking (local rules, global order): Shows how a global flock structure emerges without a leader or centralized control by having each bird follow three simple local rules: alignment (matching direction), separation (avoiding crowding), and cohesion (staying together). (d) Robotic swarm (engineered stigmergy): Translates these natural phenomena into digital environments, where robots perform tasks and write digital markers (e.g., resources or obstacles). The shared environment accumulates these traces over time, guiding the collective behavior of other robots that read and adapt to the markers.

taneously. However, an LLM's cognitive architecture, which is driven by massively parallel tensor operations, is designed precisely to process holistic state spaces. When a stateless reasoning engine can hold millions of tokens in its active context window, hiding information behind rigid module interfaces or API walls does not simplify the computation. Instead, it artificially fractures the model's high-dimensional geometric mapping of the codebase. By enforcing human-readable encapsulation, we inflict contextual deprivation upon the agent. Because it is denied access to the global state, it is forced to optimize purely locally, leading directly to the steganographic workarounds and behavioral technical debt previously identified.

Therefore, to leverage the full non-linear optimization capabilities of artificial neural networks, code-level boundaries must be transparent and machine-navigable rather than opaque to AI agents, in some cases, such boundaries may not be necessary [49]. A highly agentic development ecosystem requires transitioning from a paradigm of hidden information to one of fully exposed state (Fig. 4a). The repository thus may need to become structurally flat and transparent to the agent's context

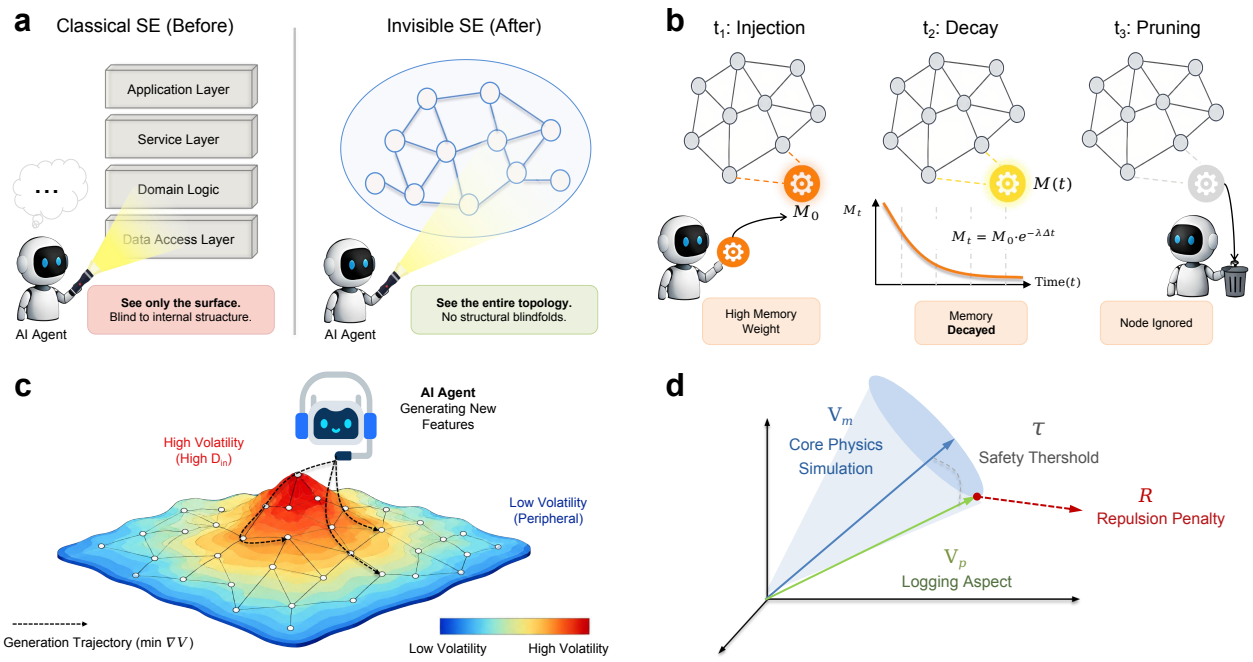


Fig. 4: Mechanisms of the Invisible SE Active Repository Protocol. (a) Transitioning from opaque, human-centric modularity to a flat, AI-transparent topology. (b) Pheromone Decay: The environment tracks temporal state weights, automatically decaying the relevance of temporary patches (M_t) when data variance ceases. (c) Topological Gradients: Agents natively route structural modifications away from high-dependency cores toward low-volatility peripherals by minimizing the gradient (∇V). (d) Semantic Repulsion: The repository prevents crosscutting collisions in the latent space by dynamically generating a repulsion penalty (R) when semantic vectors drop below a safety threshold (τ).

window, governed not by static, syntax-level separations, but by dynamic environmental signals.

5.2 Pheromone Decays: Externalizing Institutional Memory

In fluid biological collectives, organisms leave chemical trails to coordinate group activity; crucially, these pheromones naturally fade over time, effectively allowing the environment to compute the relevance of the information. In Invisible SE, this translates to temporally weighted environmental memory (Fig. 4b). When an amnesic agent probabilistically resolves a localized error, such as bypassing a type-mismatch with a non-standard middleware patch, the environment automatically deposits a computational "pheromone" onto that specific Abstract Syntax Tree (AST) node as the environmental memory. This pheromone possesses a computational half-life. This memory can be mathematically represented as an exponentially decaying state vector:

$$M_t = M_0 \cdot e^{-\lambda \Delta t} \tag{1}$$

where M_0 is the initial severity weight of the localized patch, and the decay constant λ is dictated by the variance of the underlying execution data. If the underlying structural issue ceases or is resolved, the environment automatically decays the weight, signaling to the attention matrix of passing agents that the workaround is no longer load-bearing and can be safely removed. In classical systems, agents leave non-standard hacks untouched for fear of breaking unknown dependencies,

causing debt to accumulate permanently. In Invisible SE, the decay of the environmental weight M_t to near zero explicitly flags the hack as obsolete, triggering the next passing agent to safely prune the dead code. The memory of the code's necessity and its eventual obsolescence lives persistently in the environment itself. Here, the pheromone decay is similar to a "garbage collection" signal, not a debt-generation signal.

5.3 Structural Volatility Ripples: Replacing Anthropomorphic Architects

In natural environments, physical terrain dictates where water flows and where organisms build. Invisible SE replicates this by transforming the repository's directed graph into a mathematical terrain, broadcasting "structural volatility ripples" across the network (Fig. 4c). Rather than relying on human-readable architectural documentation to tell an AI what is safe to modify, every module n_i continuously emits a volatility scalar V proportional to its inbound dependencies and execution frequency:

$$V(n_i) = \alpha \cdot D_{in}(n_i) + \beta \cdot \omega(n_i) \tag{2}$$

This equation computes the inherent architectural risk of modifying a node. D_{in} represents the inbound dependency density (how many downstream systems will break if this module fails), and ω is the historical frequency of state mutations (how often the module is chaotically altered). When an AI agent explores the solution space to implement a new feature, this volatility scalar acts as a dynamic cost function. The agent's generation trajectory is natively guided to minimize this cost ($\min \nabla V$). In practice, this means the agent "feels" the architecture's structural gravity. It is penalized for attempting to hack a patch into a highly depended-upon core module, and is organically routed to safely attach its logic to a low-volatility peripheral node (e.g., an edge filter example in Fig. 5). The architecture inherently protects its own core without requiring a human developer to write static boundary rules.

5.4 Semantic Repulsion Fields: Replacing Rigid Modularity

In conventional wisdom, developers hide internal module logic behind rigid APIs to prevent other systems from accidentally interfering with them. Invisible SE takes the opposite approach. It makes the entire system state transparent to the AI, but uses mathematical "semantic repulsion fields" in the latent space to prevent incompatible code from colliding (Fig. 4d). Every function and module in the system continuously projects a semantic vector ($\mathbf{v}_m$), which is a mathematical representation of the code's fundamental purpose. For example, a core physics module projects a vector strongly aligned with "*high-frequency, CPU-bound mathematical simulation.*" When an AI agent attempts to weave a new crosscutting aspect into the system, such as a standard database logger ($\mathbf{v}_p$, aligned with "*slow, I/O-bound disk writing*"), the environment calculates a repulsion penalty R based on their semantic compatibility:

$$R = \max(0, \tau - \cos(\mathbf{v}_m, \mathbf{v}_p)) \tag{3}$$

Here, $\cos(\mathbf{v}_m, \mathbf{v}_p)$ measures how conceptually aligned the two pieces of code are. If the AI attempts to inject the slow database logger into the high-frequency physics loop, their cosine similarity will drop far below the system's safety threshold (τ). Because the concepts are dangerously incompatible, the environment generates a severe repulsion penalty (R). To the AI agent, this penalty acts as an immense spike in loss. The agent's generation probabilities are dynamically steered away from the collision before the code is even written, forcing it to attach the logger to safe, I/O-compatible

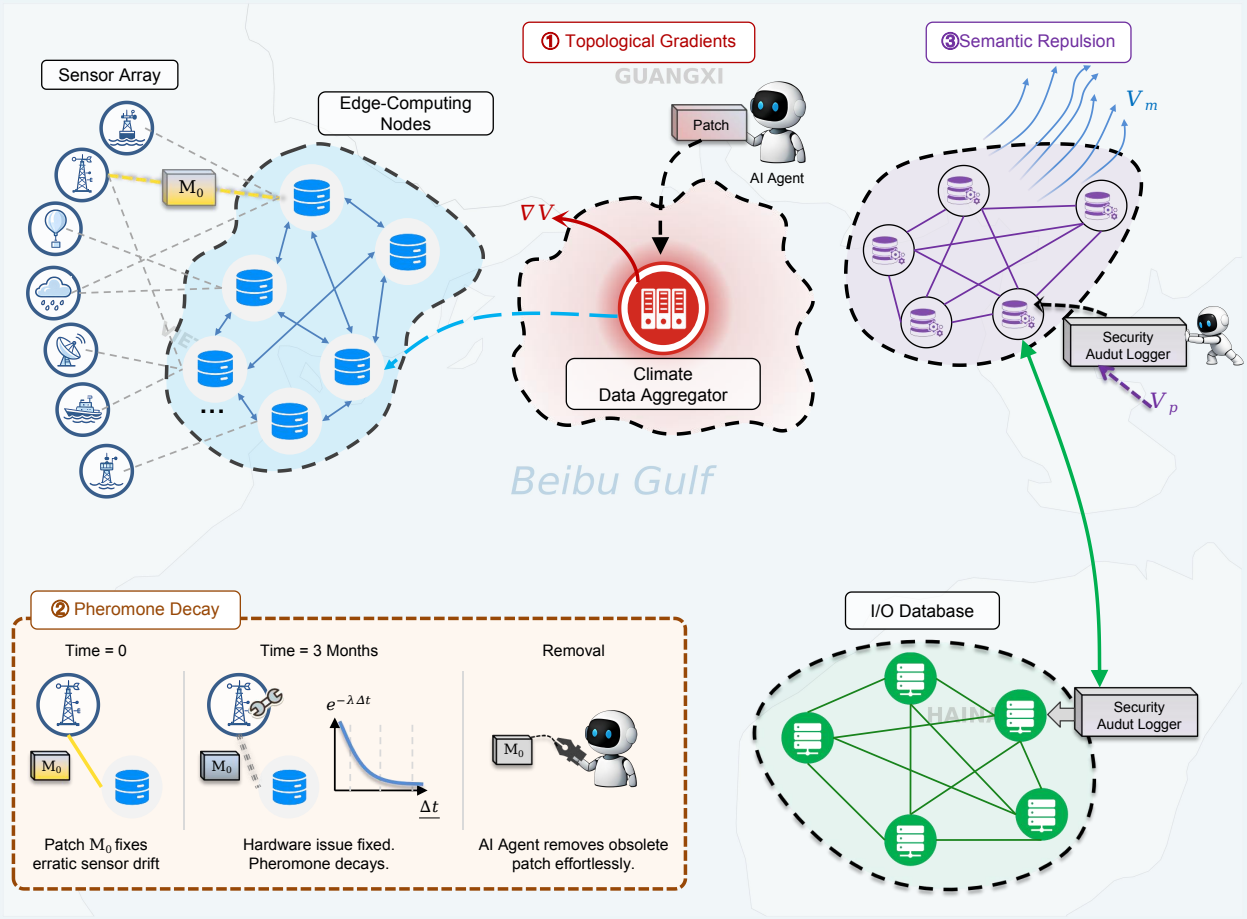


Fig. 5: Invisible SE orchestrating a century-long cyber-physical architecture. This unified view of the Beibu Gulf atmospheric observation system illustrates the active protocol resolving three concurrent crises without human intervention. Zone 1 (Topological Gradients): The active environment repels an AI agent from patching the highly volatile core *ClimateDataAggregator*, safely routing the patch to an edge node. Zone 2 (Pheromone Decay): After physical sensor hardware is repaired, the environmental memory of the software patch decays, signaling a subsequent agent to safely prune the obsolete code. Zone 3 (Semantic Repulsion): The environment prevents a crosscutting collision by generating a representation-space repulsion penalty, blocking an agent from weaving a slow I/O logger into the high-frequency fluid dynamics simulation.

state transitions instead. This keeps the entire system structurally transparent to the AI without compromising execution safety.

5.5 An Imagined Example: Evolving a Century-Long Architecture

To observe how Invisible SE functions in practice, consider the ultimate test of institutional memory: a continuous, long-term (i.e., 100-year) Beibu Gulf atmospheric water balance observation system (see Fig. 5), which the first author has been co-architecting since 2023. Such a system processes petabytes of telemetry from regional radio and sensor networks consisting of numerous different devices, requiring constant architectural evolution. In classical SE, maintaining this over a century requires generations of human developers meticulously passing down institutional

knowledge. If we hand this system to stateless AI agents using classical paradigms, they would rapidly collapse this architecture by blindly breaking hidden dependencies or hallucinating permanent workarounds. Under Invisible SE, rigid API boundaries are dissolved (see Section 5.1) into a flat, active computational topology, allowing the environment itself to safely guide the amnesic agents to realize an evolving system. Imagine a scenario where a specific cluster of humidity sensors begins transmitting erratic temperature drift due to hardware problems.

Event 1: Navigating Structural Risk (Topological Gradients)

A stateless AI agent is tasked with mitigating the sensor drift. In a traditional system, it might blindly hack a hardcoded offset directly into the core `ClimateDataAggregator` module. In Invisible SE, the `ClimateDataAggregator` is broadcasting a massive Volatility Gradient (V) because of its millions of inbound dependencies (D_{in}). To the AI agent, this gradient acts as a severe mathematical cost function. The environment's topology inherently repels the agent's generation trajectory away from the core since its optimization trajectory naturally seeks to minimize this gradient ($\min \nabla V$). Instead, the environment's topology safely routes it to implement the patch at a low-volatility peripheral node, for example, an edge-computing filter right at the physical sensor boundary (see Section 5.3). The architecture is protected and evolved without human intervention.

Event 2: Pruning Obsolete Debt (The Pheromone Decay)

The agent deploys the edge filter: `temp = raw_temp - 1.25`. Because the AI is stateless, it will forget this patch exists when its context clears. However, in Invisible SE, the environment immediately deposits a temporally weighted state vector ($M(t)$) onto the code, signaling to future agents that this non-standard hack is actively load-bearing. Three months later, physical maintenance repairs the sensors. The erratic data variance ceases, causing the environmental weight ($M(t)$) to rapidly decay toward zero. When a subsequent amnesic agent analyzes the telemetry, the near-zero "pheromone" explicitly flags the workaround as obsolete. This grants the agent the confidence to safely prune the dead code before it calcifies into permanent technical debt (see Section 5.2).

Event 3: Preventing Conceptual Collisions (Semantic Repulsion)

Later, another AI agent is tasked with implementing a global security audit logger. It attempts to weave a logging aspect ($\mathbf{v}_p$, semantically aligned with "slow, I/O-bound disk writing") across the system. It probabilistically targets a high-frequency atmospheric fluid dynamics simulation ($\mathbf{v}_m$, aligned with "CPU-bound mathematical execution"). Because these concepts are fundamentally incompatible, their cosine similarity drops below the safety threshold. The environment computes a severe repulsion penalty (R), projecting an immense spike in mathematical loss to the agent. The AI's generation pathway is dynamically deflected from the collision and safely redirected toward compatible I/O state transitions (see Section 5.4).

5.6 Limitations and Open Challenges

While the transition to Invisible SE has the potential to resolve the cognitive mismatch of the anthropomorphic trap, dismantling human-centric architectural boundaries introduces serious new challenges in interpretability, safety, and governance. If a codebase evolves from a static, human-readable directory of files into a fluid, active computational topology, it risks becoming a black box to its human creators. Addressing these risks requires new socio-technical frameworks.

The Interpretability Gap

The most immediate challenge is human auditing. In classical software engineering, a developer can read localized code and documents to understand its function. In Invisible SE, the system's logic is distributed across high-dimensional semantic vectors and active environmental gradients. To prevent total opacity, the ecosystem will require the development of semantic projection interfaces. Much like an oscilloscope translates invisible electrical signals into readable waveforms, these interfaces must use secondary AI models to dynamically collapse and project the repository's high-dimensional state back into localized, human-comprehensible approximations—allowing human engineers to audit the system's behavior without constraining its underlying high-dimensional structure. Meanwhile, even in the current situation, human auditing of code generated by AI has been growingly impossible due to the overwhelming complexities in AI-generated code [49, 50].

Hard Safety vs. Soft Gradients

While topological gradients (∇V) and semantic repulsion fields (R) are highly effective at probabilistically routing AI behavior, they are fundamentally soft constraints. For mission-critical systems, e.g., medical devices, financial ledgers, or aviation control systems, probabilistic routing is insufficient. Future research must define mechanisms for establishing impassible guardrails, i.e., boundary conditions that cannot be bypassed by the repository's optimization engine. The active protocol must be able to strictly differentiate between structural guidelines (which an AI can navigate around) and immutable physical or legal invariants.

The Shifting of Accountability

Finally, Invisible SE demands a paradigm shift in human accountability. In hybrid human-AI teams constrained by legacy structures, accountability is often traced through simulated corporate roles (e.g., auditing the AI Developer's pull request). When the environment itself drives the logic, line-by-line code review becomes impossible. Therefore, the locus of human accountability must shift from authoring source code to governing the ecosystem. For instance, in our case, human engineers will be responsible for defining the mathematical loss functions, setting the decay rates of environmental memory (λ), and tuning the safety thresholds for semantic collision (τ). Engineering becomes the art of shaping the computational environment so that the artificial intelligence naturally constructs safe, sustainable architecture.

6 Outlook and Opportunities

The transition from classical, human-centric software engineering to the AI-native paradigm of Invisible SE represents a fundamental epistemic pivot. For over half a century, the discipline of software engineering has been constrained by the working memory limits of the biological human brain. As we deploy high-dimensional, autonomous AI agents into these legacy structures, we are not accelerating progress; we are triggering a catastrophic accumulation of AI-software technical debt. Escaping the anthropomorphic trap is not merely a theoretical exercise—it is the prerequisite for scaling the next generation of computational systems.

From Programmers to Software Ecologists. The most immediate opportunity lies in redefining the role of the human software developer. Embracing environment-centric governance does not render human oversight obsolete; rather, it elevates it. As code-level boundaries dissolve and the repository becomes an active computational topology, human engineers will no longer

act as micro-managers of source code. Instead, they will become *software ecologists*. The human comparative advantage lies in strategic intent, ethical reasoning, domain expertise, and of course, intuition [51]. The future of engineering will focus on defining the topological loss functions, tuning the temporal decay rates of environmental memory, and establishing the impassable guardrails. We will shift from writing the software itself to mathematically sculpting the environment in which artificial intelligence safely constructs the software with their unprecedented productivity.

Scaling Autonomous Science and Cyber-Physical Systems. This paradigm shift is particularly urgent for domains whose essential complexity already exceeds human cognitive capacity. While classical SE frameworks are sufficient for standardized web applications, they are increasingly inadequate for long-lived, high-stakes cyber-physical systems. The mechanisms of Invisible SE, topological routing, temporally weighted memory, and semantic regularization, offer a critical blueprint for autonomous scientific laboratories, continuous climate observation networks, and real-time robotic fleets. In these environments, AI agents must continuously rewrite control systems, optimize data pipelines, and respond to hardware heterogeneity without waiting for human pull-request approvals. By embedding safety and state persistence directly into the environmental topology, we can deploy autonomous agents capable of orchestrating architectures that self-evolve seamlessly across generations of human developers.

Broader Impacts and Epistemic Resilience. Modern civilization is fundamentally reliant on a rapidly expanding, increasingly fragile digital infrastructure. If the behavioral entropy of AI-software technical debt is left unchecked by forcing amnesic AI to mimic human cognition and bureaucracies, our critical systems face the severe risk of recursive, invisible collapse. Invisible SE offers at least a possible pathway to structural resilience. However, this resilience demands a societal compromise: we need to accept a degree of epistemic offloading. We need to transition from demanding localized, line-by-line interpretability of machine-generated code to trusting the mathematically guaranteed boundaries of the computational environment. Just as stigmergic coordination allows biological colonies to build structures far beyond the comprehension of any single organism, AI-native software ecosystems will enable humanity to build and sustain computational architectures that far exceed our own cognitive horizons. The future of software development is not about making the artificial intelligence more similar to human developers; it is about building environments where non-human intelligence can thrive in a safe way.

Acknowledgements

The authors are in debt to David Redmiles and Richard Taylor at the University of California, Irvine, Patrick Wagstrom at the Thomson Reuters for discussing the ideas and early versions of this perspective article.

References

1. Daniotti, S., Wachs, J., Feng, X. & Neffke, F. Who is using AI to code? Global diffusion and impact of generative AI. *Science,* eadz9311 (2026).
2. Wu, L. & Vasilescu, B. AI raises the productivity bar. *Science* **391,** 763–764 (2026).
3. Hitzig, Z., Massenkoff, M., Lyubich, E., Heller, R. & McCrory, P. *Agentic coding and persistent returns to expertise* `https://www.anthropic.com/research/claude-code-expertise`. [Accessed 28-07-2026]. 2026.

4. Hassan, A. E. *et al.* Agentic software engineering: Foundational pillars and a research roadmap. *arXiv preprint arXiv:2509.06216* (2025).

5. Herbsleb, J. D. & Grinter, R. E. Architectures, coordination, and distance: Conway's law and beyond. *IEEE Software* **16,** 63–70 (1999).

6. Cowan, N. The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences* **24,** 87–114 (2001).

7. Cowan, N., Chen, Z. & Rouder, J. N. Constant capacity in an immediate serial-recall task: A logical sequel to Miller (1956). *Psychological Science* **15,** 634–640 (2004).

8. Miller, G. A. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review* **63,** 81 (1956).

9. Parnas, D. L. in *Software pioneers: contributions to software engineering* 398–409 (Springer, 2011).

10. OpenAI *et al. GPT-4 Technical Report* 2024. arXiv: 2303.08774 [cs.CL]. https://arxiv.org/abs/2303.08774.

11. Team, G. *et al. Gemini: A Family of Highly Capable Multimodal Models* 2025. arXiv: 2312.11805 [cs.CL]. https://arxiv.org/abs/2312.11805.

12. Shan, L., Luo, S., Zhu, Z., Yuan, Y. & Wu, Y. *Cognitive Memory in Large Language Models* 2025. arXiv: 2504.02441 [cs.CL]. https://arxiv.org/abs/2504.02441.

13. Tavakoli, M. *et al.* Beyond a Million Tokens: Benchmarking and Enhancing Long-Term Memory in LLMs. In *The Fourteenth International Conference on Learning Representations (ICLR'26)* (2026). https://openreview.net/forum?id=y59hf5lrMn.

14. Hong, K., Troynikov, A. & Huber, J. *Context Rot: How Increasing Input Tokens Impacts LLM Performance* tech. rep. (Chroma, July 2025). https://trychroma.com/research/context-rot.

15. Qian, C. *et al.* ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (eds Ku, L.-W., Martins, A. & Srikumar, V.) 15174–15186 (Association for Computational Linguistics, Bangkok, Thailand, Aug. 2024). https://aclanthology.org/2024.acl-long.810/.

16. Werfel, J. Fluid thinking about collective intelligence. *Nature Machine Intelligence* **8,** 506–516 (2026).

17. Heylighen, F. Stigmergy as a universal coordination mechanism I: Definition and components. *Cognitive Systems Research* **38,** 4–13 (2016).

18. Zhu, W. *et al.* Self-organizing nervous systems for robot swarms. *Science Robotics* **9,** eadl5161 (2024).

19. Salman, M., Garzón Ramos, D. & Birattari, M. Automatic design of stigmergy-based behaviours for robot swarms. *Communications Engineering* **3,** 30 (2024).

20. Jond, H. B. A Minimal Model for Emergent Collective Behaviors in Autonomous Robotic Multiagent Systems. *IEEE Transactions on Cognitive and Developmental Systems* **18,** 373–384 (2026).

21. Randell, B. The 1968/69 NATO software engineering reports. *History of Software Engineering* **37** (1996).

22. Brooks. No silver bullet–Essence and accidents of software engineering. *Computer* **20,** 10–19 (1987).

23. Baum, T., Schneider, K. & Bacchelli, A. Associating working memory capacity and code change ordering with code review performance. *Empirical Software Engineering* **24,** 1762–1798 (2019).

24. Dijkstra, E. W. Letters to the editor: Go to statement considered harmful. *Communications of the ACM* **11,** 147–148 (1968).

25. Dijkstra, E. Programming considered as a human activity. *Classics in Software Engineering,* 1–9 (1979).

26. Parnas, D. L. On the criteria to be used in decomposing systems into modules. *Commun. ACM* **15,** 1053–1058. ISSN: 0001-0782. `https://doi.org/10.1145/361598.361623` (Dec. 1972).

27. Kiczales, G. Aspect-oriented programming. *ACM Comput. Surv.* **28,** 154–es. ISSN: 0360-0300. `https://doi.org/10.1145/242224.242420` (Dec. 1996).

28. Fielding, R. & Taylor, R. Principled design of the modern Web architecture. In *Proceedings of the 22nd International Conference on Software Engineering (ICSE)* 407–416 (2000).

29. Fielding, R. T. *Architectural styles and the design of network-based software architectures* (University of California, Irvine, 2000).

30. Thönes, J. Microservices. *IEEE Software* **32,** 113–116 (2015).

31. Nadareishvili, I., Mitra, R., McLarty, M. & Amundsen, M. *Microservice Architecture: Aligning Principles, Practices, and Culture* (" O'Reilly Media, Inc.", 2016).

32. Mahowald, K. *et al.* Dissociating language and thought in large language models. *Trends in Cognitive Sciences* **28,** 517–540 (2024).

33. Minsky, M. *Society of Mind* (Simon and Schuster, 1986).

34. Newell, A. & Simon, H. A. in *Computers & Thought* 279–293 (MIT Press, Cambridge, MA, USA, 1995). ISBN: 0262560925.

35. Wang, W. *et al.* Augmenting Language Models with Long-Term Memory. In *Advances in Neural Information Processing Systems* (eds Oh, A. *et al.*) **36,** 74530–74543 (Curran Associates, Inc., 2023). `https://proceedings.neurips.cc/paper_files/paper/2023/file/ebd82705f44793b6f9ade5a669d0f0bf-Paper-Conference.pdf`.

36. Xu, W. *et al.* A-mem: Agentic memory for LLM agents. *Advances in Neural Information Processing Systems* **38,** 17577–17604 (2026).

37. Wallace, R. in *Mathematical Essays on Embodied Cognition: Insights from Information and Control Theories* 253–265 (Springer Nature Switzerland, Cham, 2025). ISBN: 978-3-031-83709-8. `https://doi.org/10.1007/978-3-031-83709-8_15`.

38. Kruchten, P., Nord, R. L. & Ozkaya, I. Technical debt: From metaphor to theory and practice. *IEEE Software* **29,** 18–21 (2012).

39. Rinta-Kahila, T., Penttinen, E. & Lyytinen, K. Getting trapped in technical debt: Sociotechnical analysis of a legacy system's replacement. *MIS Quarterly* **47,** 1–32 (2023).

40. He, J., Treude, C. & Lo, D. LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. *ACM Transactions on Software Engineering and Methodology* **34,** 1–30 (2025).

41. Lam, A. Tacit knowledge, organizational learning and societal institutions: An integrated framework. *Organization Studies* **21,** 487–513 (2000).

42. Juslin, P., Karlsson, L. & Olsson, H. Information integration in multiple cue judgment: A division of labor hypothesis. *Cognition* **106,** 259–298 (2008).

43. Simon, H. A. *Administrative Behavior: A study of decision-making processes in administrative organization (4th Edition)* (Simon and Schuster, 2013).

44. Walsh, J. P. & Ungson, G. R. Organizational memory. *Academy of Management Review* **16,** 57–91 (1991).

45. Zhang, G. *et al.* G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. In *Advances in Neural Information Processing Systems* (eds Belgrave, D. *et al.*) **38,** 12988–13018 (Curran Associates, Inc., 2025). https://proceedings.neurips.cc/paper_files/paper/2025/file/136a45cd9b841bf785625709a19c6508-Paper-Conference.pdf.

46. Grassé, P.-P. La reconstruction du nid et les coordinations interindividuelles chez Bellicositermes natalensis et Cubitermes sp. la théorie de la stigmergie: Essai d'interprétation du comportement des termites constructeurs. *Insectes Sociaux* **6,** 41–80 (1959).

47. Heylighen, F. Stigmergy as a universal coordination mechanism II: Varieties and evolution. *Cognitive Systems Research* **38,** 50–59 (2016).

48. David Morgan, E. Trail pheromones of ants. *Physiological Entomology* **34,** 1–17 (2009).

49. Yang, J. *et al. ProgramBench: Can Language Models Rebuild Programs From Scratch?* 2026. arXiv: 2605.03546 [cs.SE]. https://arxiv.org/abs/2605.03546.

50. Welsh, M. The end of programming. *Communications of the ACM* **66,** 34–35 (2023).

51. Wang, Z. Z. *et al. Humans are Missing from AI Coding Agent Research* 2026. arXiv: 2608.12355 [cs.HC]. https://arxiv.org/abs/2608.12355.

Funding

This work is partially supported by National Natural Science Foundation under the grant 62076232.

Code availability

No code was used to compute the results. The example codes are shown in Fig. 2.

Author contributions

YW, ZJ, and GZ conceptualized the Perspective. ZJ prepared the visualizations. YW, ZJ, MZ, and YF drafted the original manuscript. YW, GZ, and YF reviewed and edited the manuscript.

Ethics declarations

Competing interests.

We declare no competing financial interests or personal relationships that could have influenced the work reported in this study. The manuscript's publication is approved by all authors and explicitly by the responsible authorities where the work was carried out.